

제 4회 ETRI 휴먼이해 인공지능 논문경진대회

[모델 설명서]



2025.09

팀 소개 및 분석 환경

◎ 팀명 : 단머스 (단기 머신러닝 스터디)

◎ 팀원 : 현종열, 변유철, 박경근

제 4회 ETRI 휴먼이해 인공지능 논문경진대회

알고리즘 | 정형 | 라이프로그 | 논문 | 분류 | Macro F1-Score

₩ 상금 : 770만원

🕒 2025.04.21 ~ 2025.07.23 09:59

+ Google Calendar

👤 1,034명 📅 마감



참여중

대회 URL

<https://dacon.io/competitions/official/236468/overview/description>

분석 코드

<https://github.com/dmskorea/project8-The-4th-ETRI-AI-Human-Understanding-Competition/tree/main/submissions>

[분석 서버 사양]

Google Colab Pro+ , A100 VRAM 40G x 1개

목차

I. PATH 경로 필요한 파일 목록

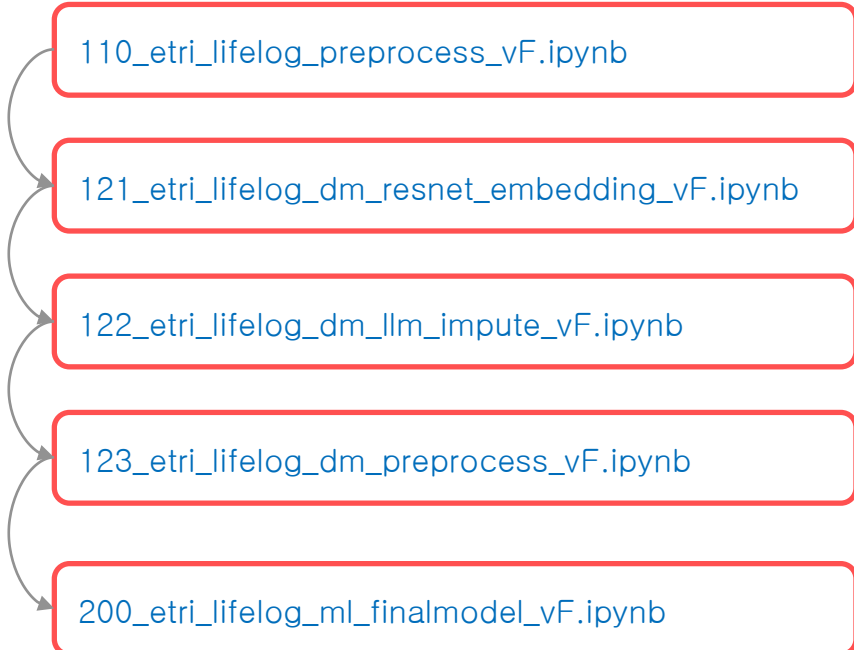
II. 데이터 전처리 (기본 전처리)

III. Resnet embedding Features

IV. LLM을 활용한 결측값 추정 (논문 기재 내용)

V. 기본 데이터셋 + 추가 파생변수

VI. 모델 학습 및 submission 파일 생성



110_etri_lifelog_preprocess_vF.ipynb

121_etri_lifelog_dm_resnet_embedding_vF.ipynb

122_etri_lifelog_dm_llm_impute_vF.ipynb

123_etri_lifelog_dm_preprocess_vF.ipynb

200_etri_lifelog_ml_finalmodel_vF.ipynb

PATH 경로 필요한 파일 목록

© 데이터 출처 : <https://dacon.io/competitions/official/236468/data>

```
# 1
mACStatus = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mACStatus.parquet')
mActivity = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mActivity.parquet')
mAmbience = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mAmbience.parquet')
mBle = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mBle.parquet')
mGps = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mGps.parquet')
mLight = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mLight.parquet')
mScreenStatus = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mScreenStatus.parquet')
mUsageStats = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mUsageStats.parquet')
mWifi = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_mWifi.parquet')
wHr = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_wHr.parquet')
wLight = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_wLight.parquet')
wPedo = pd.read_parquet(f'{PATH}/ETRI_lifelog_dataset/ch2025_data_items/ch2025_wPedo.parquet')

# 2
train = pd.read_csv(f'{PATH}/ETRI_lifelog_dataset/ch2025_metrics_train.csv')
test = pd.read_csv(f'{PATH}/ETRI_lifelog_dataset/ch2025_submission_sample.csv')
```

◎ 코드명 : [110_etri_lifelog_dm_preprocess_vF.ipynb](#)

◎ 코드 설명 : 기본 파생변수 생성을 위한 전처리 코드

◎ 사전 작업 :

1) PATH 설정

경진대회에서 제공 된 파일이
저장된 경로로 변경하세요.

```
PATH = '/content/drive/MyDrive/data/ch2025_data_items/share/submissions/input'
```

2) 라이브러리 설치

◎ OUTPUT 파일

```
train_63775_vF.parquet  
test_63775_vF.parquet
```

개별 데이터별 전처리 및 파생 변수:

- **mACStatus (충전상태):**
 - 충전 비율, 총 충전 시간, 상태 전환 횟수, 평균/최대 충전 지속 시간 (일별, 수면 시간대별).
- **mActivity (추정행동):**
 - 걷기/차량/활동 시간 (일별, 수면 시간대별).
 - 활동 및 MET(대사 증가) 값의 시간대별 통계 (표준편차, 합계 등)
- **mAmbience (주변소리):**
 - 고유 소리 라벨 수, 코골이(snor) 발생 횟수 (활동/수면 시간대별)
- **mBle (블루투스):**
 - RSSI(신호 강도) 평균/최소/최대, 알 수 없는/기타 기기 비율 (근무/퇴근 후/수면 시간대별)
- **mGps (GPS 위치):**
 - 걷기/조깅/차량 이동 시간, 평균/최대/최소 속도, 이동 거리 (활동/수면 시간대별)
 - 운동 플래그, 첫 기상 시간(분).
- **mLight (주변 밝기):**
 - 일별 밝기 통계 (평균, 표준편차, 최대, 최소, 밤/낮 평균, 밤 비율).
 - 수면/기상 시간, 수면 지속 시간 (밝기 기반 추정).
 - 불 끈 시간 추정.

개별 데이터별 전처리 및 파생 변수:

- **mScreenStatus (화면 사용):**
 - 수면/기상 시간, 수면 지속 시간 (화면 사용 기반 추정).
- **mUsageStats (앱 사용):**
 - 특정 앱(통화, 메신저, YouTube, NAVER 등) 사용 시간.
 - 고유 앱 개수, 총 화면 사용 시간, 평균 대비 화면 사용량(%).
 - 시간대별(취침 전, 활동 시간) 앱 사용 통계.
- **mWifi (주변 Wifi):**
 - 스캔 횟수, 고유 BSSID 수, RSSI 통계, 강한 신호 비율, 빈 스캔 횟수, 최다 탐지 BSSID, 시간 범위 (시간대별).
- **wHr (심박동수):**
 - 심박수 평균/최소/최대/표준편차, 고심박수 합계 (활동/수면 시간대별).
- **wLight (스마트워치 밝기):**
 - 밝기 평균/최소/최대/표준편차, 첫 취침/기상 시간(분) (활동/수면 시간대별).
- **wPedo (걸음수):**
 - 총 걸음 수, 이동 거리, 소모 칼로리 (활동/수면 시간대별).
- **AwakeBlocks (야간 각성):**
 - 야간(0-6시) 각성 시간(분), 각성 블록 횟수 (mLight, wLight 기반).

Resnet embedding Features

코드

분석



◎ 코드명 : [121_etri_lifelog_dm_resnet_embedding_vF.ipynb](#)

◎ 코드 설명 : Resnet50 embedding features 생성 (with PCA 활용)

◎ 사전 작업 :

1) PATH 설정

경진대회에서 제공 된 파일이
저장된 경로로 변경하세요.

```
PATH = '/content/drive/MyDrive/data/ch2025_data_items/share/submissions/input'
```

◎ Info : Google Colab Pro+ 기준, 작업 중간 RAM부족으로 세션 다운 됨 → 세션 재시작 후 모두 재실행 반복 x 5~6회 작업 필요

1차

```
# train: 450
# test: 250
# 전체 데이터 : 700
# 남은 샘플수 : 700
user: id05, date: 2024-09-02: 26% 184/700 [11:00<32:30, 3.78s/it]
```

사용 가능한 RAM을 모두 사용한 후 세션이 다운되었습니다.

Google Colab Pro+ 기준
26% 에서 RAM부족으로 세션
다운 됨

세션 재시작 후
모두재실행하면 마지막 작업
이후로 **이어서** 작업 진행 됨

2차

```
# train: 450
# test: 250
# 전체 데이터 : 700
# 남은 샘플수 : 516
user: id06, date: 2024-06-03: 8% 39/516 [02:19<28:34, 3.59s/it]
```


Resnet embedding Features

코드

분석



© 코드명 : [121_etri_lifelog_dm_resnet_embedding_vF.ipynb](#)

© OUTPUT 파일

img_features_ch5 sleeptime_resnet50_5.csv

ResNet 피쳐 추출 핵심 과정

- **시계열 데이터 이미지 변환:**
 - 라이프로그 시계열 데이터 처리 (특정 시간대(00-06시) 데이터 집중)
 - PNG 이미지 파일 생성.
 - 사용자-날짜별 시계열 채널 그래프화.
- **이미지 전처리:**
 - 생성 이미지 평균/표준편차 계산.
 - ResNet 입력 전 정규화.
 - 이미지 리사이즈(500x500).
 - 평균/표준편차 기반 정규화 적용.
- **ResNet 피쳐 추출:**
 - timm 라이브러리 활용.
 - resnet50 사전 학습 모델 로드.
 - 모델 분류 레이어 제거.
 - 피쳐 추출기 역할 전환.
 - 전처리 이미지 입력.
 - 특징 벡터(임베딩) 획득.
- **피쳐 저장:**
 - 추출 피쳐 PCA 차원 축소(선택) 및 csv 파일 저장

LLM을 활용한 결측값 추정 (논문 기재 내용)

코드

분석



◎ 코드명 : 122_etri_lifelog_dm_llm_impute_vF.ipynb

◎ 코드 설명 : LLM을 활용한 결측값 추정 파생변수 생성

◎ 사전 작업 :

1) PATH 설정

```
PATH = '/content/drive/MyDrive/data/ch2025_data_items/share/submissions/input'
```

2) 라이브러리 설치 (※ 메모리 이슈로 설치 후 세션 재시작 필요)

vllm 설치

세션 다시시작!

Qwen/Qwen3-8B 다운로드

세션 다시시작!

Qwen/Qwen3-8B 로컬에서 로딩

```
try:
    from vllm import LLM, SamplingParams
except:
    !pip install -U langchain-community >/dev/null
    !pip install bitsandbytes >/dev/null
    !pip install -U transformers accelerate >/dev/null
    !pip install faiss-gpu-cu12 --no-deps >/dev/null
    !pip install datasets >/dev/null
    !pip install vllm >/dev/null
    !pip install --upgrade transformers >/dev/null
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import os

# 구글 드라이브 경로
drive_path = "/content/drive/MyDrive/models2"

# 모델명
model_id = "Qwen/Qwen3-8B"

# 저장 경로
save_path = os.path.join(drive_path, model_id)

# 토큰라이저와 모델 다운로드 (Drive에 자동 저장됨)
tokenizer = AutoTokenizer.from_pretrained(model_id, cache_dir=save_path)
model = AutoModelForCausalLM.from_pretrained(model_id, cache_dir=save_path)
```

```
import os
os.environ["VLLM_WORKER_MULTIPROC_METHOD"] = "spawn"

from google.colab import drive, files
drive.mount('/content/drive')

from vllm import LLM, SamplingParams

# 경로
drive_path = "/content/drive/MyDrive/models2"

# 모델명
model_id = 'Qwen/Qwen3-8B'

# vllm
llm = LLM(
    model=f"{drive_path}/{model_id}",
    tokenizer=f"{drive_path}/{model_id}",
    tensor_parallel_size=1,
    dtype="bfloat16",
    load_format="auto",
    gpu_memory_utilization=0.8,
    max_model_len=38960,
    enforce_eager=True, |
```

◎ OUTPUT 파일

mScreenStatus_llm_20250607_v4

"""

vllm 설치 코드

"""

```
!pip install -U langchain-community >/dev/null
!pip install bitsandbytes >/dev/null
!pip install -U transformers accelerate >/dev/null
!pip install faiss-gpu-cu12 --no-deps >/dev/null
!pip install datasets >/dev/null
!pip install vllm >/dev/null
!pip install --upgrade transformers >/dev/null
```

설치 후 세션 재시작

```
"""
LLM 다운로드 코드
"""

from transformers import AutoModelForCausalLM, AutoTokenizer
import os

# 구글 드라이브 경로
drive_path = "/content/drive/MyDrive/models2"

# 모델명
model_id = "Qwen/Qwen3-8B"

# 저장 경로
save_path = os.path.join(drive_path, model_id)

# 토큰라이저와 모델 다운로드 (Drive에 자동 저장됨)
tokenizer = AutoTokenizer.from_pretrained(model_id, cache_dir=save_path)
model = AutoModelForCausalLM.from_pretrained(model_id, cache_dir=save_path)

print(f"모델과 토큰라이저가 저장되었습니다: {save_path}")
```

다운 후 세션 재시작

"""

다운로드 된 LLM 로드

"""

```
import os
os.environ["VLLM_WORKER_MULTIPROC_METHOD"] = "spawn"

from vllm import LLM, SamplingParams

# 모델 저장 경로
drive_path = "/content/drive/MyDrive/models2/"

# 모델명
model_id = 'Qwen/Qwen3-8B'

# vllm
llm = LLM(
    model=f"{drive_path}{model_id}",
    tokenizer=f"{drive_path}{model_id}",
    tensor_parallel_size=1,
    dtype="bfloat16",
    load_format="auto",
    gpu_memory_utilization=0.8,
    max_model_len=38960,
    enforce_eager=True,
)

from transformers import AutoTokenizer, AutoModelForCausalLM

# Tokenizer 로드
tokenizer = AutoTokenizer.from_pretrained("/content/drive/MyDrive/models2/Qwen/Qwen3-8B", trust_remote_code=True)
```

LLM 결측치 보간 핵심 과정:

• LLM 준비:

- vllm 라이브러리 사용.
- 사전 학습된 LLM 모델(Qwen/Qwen3-8B 등) 로드 및 토큰나이저 설정

• 데이터 전처리 및 LLM 입력 준비:

- mScreenStatus 데이터 로드.
- 화면 사용 패턴 기반 취침시간, 기상시간, 수면시간 초기 추정 및 야간/새벽 시간대 필터링
- 짧은 각성/수면 블록 제거로 추정치 정제.
- 시간 값을 소수점 형태로 변환.
- LLM 지침(system_message) 및 사용자 질의(user_message) 프롬프트 구성.
 - 지침: 데이터 분석 전문가 역할, 데이터 설명, 결측치 보간 규칙(유효 범위, 기존 값 유지, 모든 결측치 채우기), 금지 사항 포함.
 - 질의: 특정 사용자 데이터, 통계 정보, 도메인 지식(요일별, 계절별, 전일 상태 영향) 제공.

• LLM 보간 실행:

- 각 사용자별 반복 처리.
- 동적으로 구성된 프롬프트를 LLM에 전달.
- 재시도 메커니즘: LLM 출력에 결측치가 많거나 생성된 행 수가 부족할 경우 최대 3회 재시도 (품질 관리).
- LLM 텍스트 출력 파싱 (탭 구분 테이블).

• 보간 후 처리:

- LLM 보간된 취침시간, 기상시간을 원본 데이터에 병합 (결측치만 채움).
- (보간된) 취침/기상 시간을 기반으로 수면시간 재계산.
- 업데이트된 수면 관련 데이터로 추가 시계열 파생 변수(차이, 비율, 시차, 이동 평균/표준편차) 생성.
- 최종 처리 데이터 및 중간 결과 저장.

◎ 코드명 : [123_etri_lifelog_dm_preprocess_vF.ipynb](#)

◎ 코드 설명 : 기본 데이터셋에, LLM-based imputation Features + Resnet embedding Features 변수 추가

◎ 사전 작업 :

1) PATH 설정

경진대회에서 제공 된 파일이
저장된 경로로 변경하세요.

```
PATH = '/content/drive/MyDrive/data/ch2025_data_items/share/submissions/input'
```

2) 라이브러리 설치

◎ OUTPUT 파일

train_hjy_0603_vF.parquet
Test_hjy_0603_vF.parquet

추가 파생변수 :

mScreenStatus 컬럼, LLM으로 결측값 추정 후, 아래 변수 추가적으로 생성

```
feats = ['sleep_time', 'wake_time', 'sleep_duration_min', 'avg_sleep_time', 'avg_wake_time', 'avg_sleep_duration', 'sleep_time_diff',  
'wake_time_diff', 'sleep_duration_diff', 'sleep_time_ratio', 'wake_time_ratio', 'sleep_duration_ratio', 'sleep_time_lag1', 'wake_time_lag1',  
'sleep_duration_lag1', 'sleep_time_diff_lag1', 'wake_time_diff_lag1', 'sleep_duration_diff_lag1', 'sleep_time_ratio_lag1', 'wake_time_ratio_lag1',  
'sleep_duration_ratio_lag1', 'sleep_time_lag2', 'wake_time_lag2', 'sleep_duration_lag2', 'sleep_time_diff_lag2', 'wake_time_diff_lag2',  
'sleep_duration_diff_lag2', 'sleep_time_ratio_lag2', 'wake_time_ratio_lag2', 'sleep_duration_ratio_lag2', 'sleep_time_mean2d',  
'wake_time_mean2d', 'sleep_duration_min_mean2d', 'sleep_time_diff_mean2d', 'wake_time_diff_mean2d', 'sleep_duration_diff_mean2d',  
'sleep_time_ratio_mean2d', 'wake_time_ratio_mean2d', 'sleep_duration_ratio_mean2d', 'sleep_time_std2d', 'wake_time_std2d',  
'sleep_duration_min_std2d', 'sleep_time_diff_std2d', 'wake_time_diff_std2d', 'sleep_duration_diff_std2d', 'sleep_time_ratio_std2d',  
'wake_time_ratio_std2d', 'sleep_duration_ratio_std2d', 'sleep_time_mean3d', 'wake_time_mean3d', 'sleep_duration_min_mean3d',  
'sleep_time_diff_mean3d', 'wake_time_diff_mean3d', 'sleep_duration_diff_mean3d', 'sleep_time_ratio_mean3d', 'wake_time_ratio_mean3d',  
'sleep_duration_ratio_mean3d', 'sleep_time_std3d', 'wake_time_std3d', 'sleep_duration_min_std3d', 'sleep_time_diff_std3d',  
'wake_time_diff_std3d', 'sleep_duration_diff_std3d', 'sleep_time_ratio_std3d', 'wake_time_ratio_std3d', 'sleep_duration_ratio_std3d',  
'sleep_time_mean5d', 'wake_time_mean5d', 'sleep_duration_min_mean5d', 'sleep_time_diff_mean5d', 'wake_time_diff_mean5d',  
'sleep_duration_diff_mean5d', 'sleep_time_ratio_mean5d', 'wake_time_ratio_mean5d', 'sleep_duration_ratio_mean5d', 'sleep_time_std5d',  
'wake_time_std5d', 'sleep_duration_min_std5d', 'sleep_time_diff_std5d', 'wake_time_diff_std5d', 'sleep_duration_diff_std5d',  
'sleep_time_ratio_std5d', 'wake_time_ratio_std5d', 'sleep_duration_ratio_std5d', 'sleep_time_mean7d', 'wake_time_mean7d',  
'sleep_duration_min_mean7d', 'sleep_time_diff_mean7d', 'wake_time_diff_mean7d', 'sleep_duration_diff_mean7d', 'sleep_time_ratio_mean7d',  
'wake_time_ratio_mean7d', 'sleep_duration_ratio_mean7d', 'sleep_time_std7d', 'wake_time_std7d', 'sleep_duration_min_std7d',  
'sleep_time_diff_std7d', 'wake_time_diff_std7d', 'sleep_duration_diff_std7d', 'sleep_time_ratio_std7d', 'wake_time_ratio_std7d',  
'sleep_duration_ratio_std7d', 'weekday_avg_sleep', 'sleep_duration_weekday_avg_diff', 'sleep_duration_weekday_avg_div']
```

◎ 코드명 : [200_etri_lifelog_ml_finalmodel_vF.ipynb](#)

◎ 코드 설명 : 최종 모델 학습 및 submission 파일 생성

◎ 사전 작업 :

1) PATH 설정

경진대회에서 제공 된 파일이
저장된 경로로 변경하세요.

```
PATH = '/content/drive/MyDrive/data/ch2025_data_items/share/submissions/input'
```

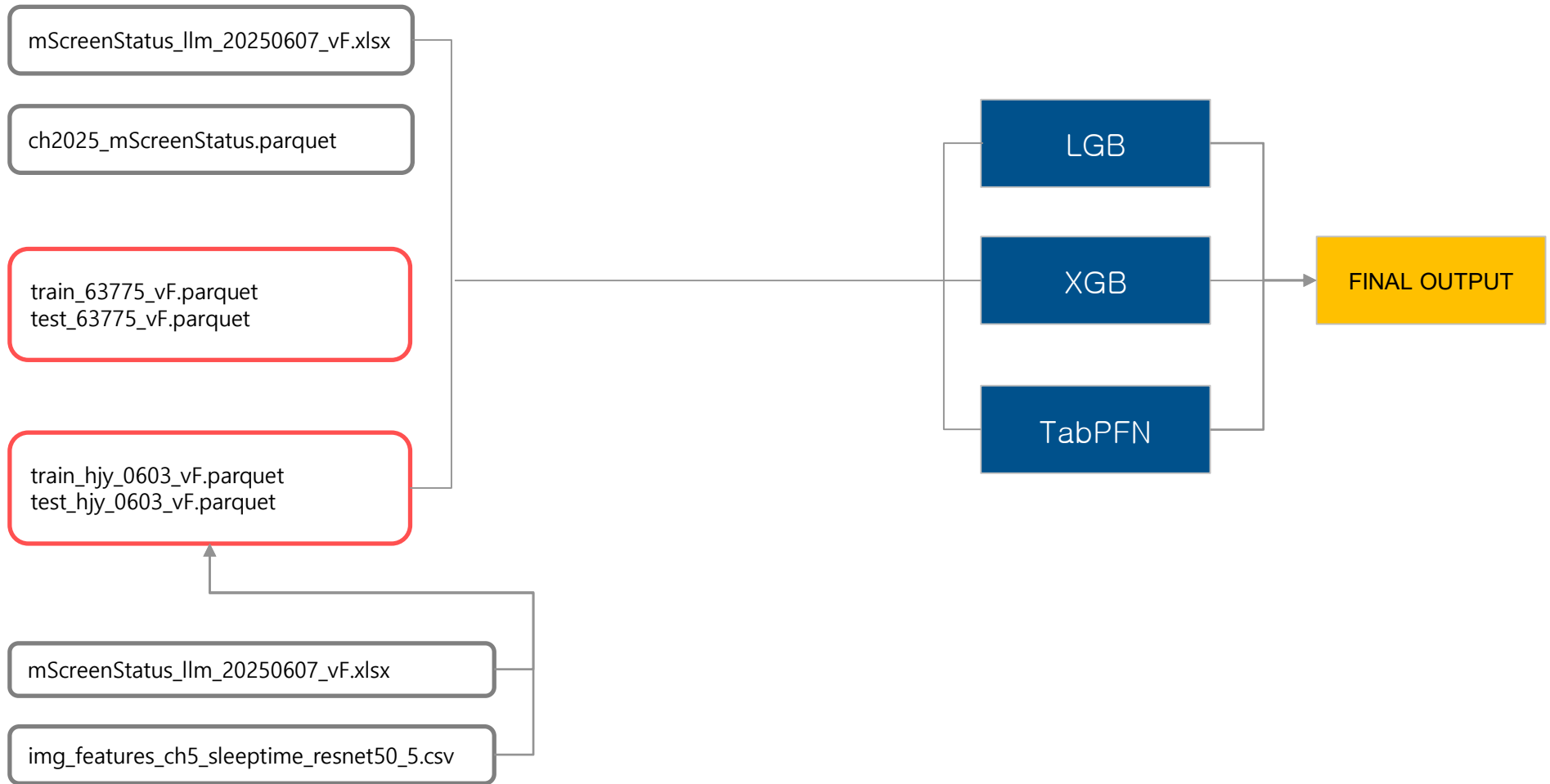
2) 라이브러리 설치

```
# 라이브러리 설치
! pip install haversine >/dev/null
! pip install optuna >/dev/null
! pip install category_encoders >/dev/null
! pip install tabPFN >/dev/null
! pip install catboost >/dev/null
! pip install torchmetrics >/dev/null
```

◎ OUTPUT 파일

submission_top1_0.7218.csv

[참고] 모델 예측에 필요한 INPUT 데이터 리스트 & 모델 구조



◎ 코드명 : 200_etri_lifelog_ml_finalmodel_vF.ipynb

◎ 코드 설명 : 최종 모델 학습 및 submission 파일 생성

◎ 모델 학습 :

1) 모델링 전략 (Modeling Strategy)

```
# ... (생략) ...
for col in targets_binary: # Q1, Q2, Q3, S2, S3 루프
    # ... (생략) ...
    # xfeatures1 (SHAP 기반, Q3에만 적용)
    if col in ['Q3']:
        model = XGBClassifier(**xgb_params)
        # ... (생략) ...
        shap_values = explainer.shap_values(X_train)
        # ... (생략) ...
        xfeatures1 = shap_df.head(15)['feature'].tolist()
    else:
        xfeatures1 = []

    # xfeatures2 (상관관계 기반)
    correlations = X.select_dtypes(include=['number']).corrwith(y)
    sorted_correlations = correlations.abs().sort_values(ascending=False)
    xfeatures2 = sorted_correlations[sorted_correlations>0.1].index.tolist()

    # 최종 피쳐셋 조합
    xfeatures_dict[col] = [i for i in X.columns.tolist() if i in set(xfeatures1+xfeatures2)]

    # 선택된 피쳐로 데이터셋 재생성
    X_valid = train_df.loc[train_df['pk'].isin(valid_ids['pk']),xfeatures_dict[col]].reset_index(drop=True).copy()
    X_train = train_df.loc[~train_df['pk'].isin(valid_ids['pk']),xfeatures_dict[col]].reset_index(drop=True).copy()
    # ... (생략) ...
```

- Q3를 예측할 때는 SHAP으로 피쳐 중요도를 계산해 상위 15개를 선택하고(xfeatures1), 타겟 변수와 상관관계가 0.1 이상인 피쳐들(xfeatures2)을 추가로 선택합니다.
- 나머지 타겟들은 상관관계 기반으로만 피쳐를 선택합니다.
- 이렇게 각 타겟(col)마다 다르게 선택된 피쳐 조합(xfeatures_dict[col])을 사용해 모델을 학습시켜 예측 정확도를 높입니다.

◎ 코드명 : [200_etri_lifelog_ml_finalmodel_vF.ipynb](#)

◎ 코드 설명 : 최종 모델 학습 및 submission 파일 생성

◎ 모델 학습 :

2) 3가지 모델 앙상블 (Ensemble of 3 Models)

```
# ... (생략) ...
# Train LightGBM
lgb_model = LGBMClassifier(**lgb_params)
lgb_model.fit(X_train, y_train)

# Train XGBoost
xgb_model = XGBClassifier(**xgb_params)
xgb_model.fit(X_train, y_train)

# Train TabPFN
tabpfn_model = TabPFNClassifier(**tabpfn_params)
tabpfn_model.fit(X_train, y_train)

# 검증 데이터셋에 대한 예측 확률 계산
lgb_pred_valid = lgb_model.predict_proba(X_valid)[ :, 1]
xgb_pred_valid = xgb_model.predict_proba(X_valid)[ :, 1]
tab_pred_valid = tabpfn_model.predict_proba(X_valid.values)[ :, 1]

# 가중 평균으로 앙상블
pred_valid = (lgb_A * lgb_pred_valid + xgb_B * xgb_pred_valid + tab_C * tab_pred_valid > 0.5).astype(int)
```

- 세 모델을 각각 학습시키고, 검증 데이터(X_valid)에 대한 예측 확률값을 얻습니다.
- 초기 가중치(lgb_A=0.333, xgb_B=0.334, tab_C=0.333)를 사용하여 예측값들을 합산하고, 임계값(0.5)을 기준으로 최종 클래스를 결정합니다.

◎ 코드명 : [200_etri_lifelog_ml_finalmodel_vF.ipynb](#)

◎ 코드 설명 : 최종 모델 학습 및 submission 파일 생성

◎ 모델 학습 :

3) 최적의 앙상블 가중치 탐색 (Finding Optimal Weights)

```
# 0.0부터 1.0까지 0.1 스텝으로 가능한 모든 가중치 조합 생성
step = 0.1
candidates = np.arange(0, 1.1, step)

for lgb_A, xgb_B, tab_C in product(candidates, repeat=3):
    total = lgb_A + xgb_B + tab_C
    if np.isclose(total, 1.0): # 세 가중치의 합이 1.0인 경우만 테스트
        weights = (lgb_A, xgb_B, tab_C)
        val_scores = []

        # Binary targets
        for col in targets_binary:
            # ... (가중치를 적용하여 F1 점수 계산)
            val_scores.append(f1_score(preds['true'], (blended > 0.5).astype(int), average='macro'))

        # Multiclass target
        # ... (가중치를 적용하여 F1 점수 계산)
        val_scores.append(f1_score(preds['true'], np.argmax(blended, axis=1), average='macro'))

        best_weights.append(weights)
        best_scores.append(np.mean(val_scores)) # 6개 타겟의 평균 F1 점수 저장
```

- 세 가중치의 합이 1이 되는 모든 조합(예: 0.1, 0.2, 0.7)을 대상으로 루프를 돌립니다.
- 각 조합을 검증 데이터셋의 예측값에 적용하여 6개 타겟의 평균 F1 점수를 계산합니다.
- 가장 높은 평균 F1 점수를 기록한 가중치 조합을 찾아 저장해 둡니다.

최종 예측 모델:

- 3개 모델 앙상블:

- LightGBM: Gradient Boosting 모델.
- XGBoost: Gradient Boosting 모델.
- TabPFN: 소규모 테이블 데이터 특화 Transformer 모델.

모델 학습 및 검증:

- 검증 전략:

- 전체 학습 데이터 8.9% (40개 샘플) 검증 세트 분리 및 참가자 별 4개씩 균등 추출.
- 테스트 데이터와 분포 유사성 확보.

- 결측치 대체 방법론 비교:

- 4가지 시나리오: (1)결측치 그대로 사용, (2)평균값 대체, (3)KNN 대체, (4)LLM 대체.
- 동일 3개 모델 앙상블로 예측 성능(F1-Macro) 측정.

성능 평가:

- 전체 성능:

- LLM 기반 대체: F1 0.7218 (최고).
- 결측치 그대로 사용 대비 5.5% 성능 향상.

- 지표별 성능:

- 수면 효율(S2) 지표: LLM 대체 F1 0.775 (압도적).
- 다른 방법론 대비 약 10%p 향상.
- LLM의 문맥 이해 및 논리적 추론 능력 입증.

- 앙상블 가중치 최적화:

- (LGB:0.5, XGB:0.2, TabPFN:0.3) 가중치에서 최고 성능.